

JHQ-GPU: Representation-Driven GPU Execution and Workload-Adaptive Hierarchical Refinement

Rui Luo and
Mengxuan Zhang^[0000-0001-8155-4942]

Australian National University, Canberra, Australia
u8076655@anu.edu.au, mengxuan.zhang@anu.edu.au

Abstract. JHQ scores approximate nearest neighbours in two stages: a compact primary code over many candidates, then a per-dimension residual code over a small survivor set. Porting it to a GPU raises two questions that byte counting does not answer. First, how should the primary representation execute? JHQ’s primary codebook is Cartesian, so its 256-entry query table separates exactly into two 16-entry tables; full-table state grows as $256M$ entries against $32M$ for the split, an $8\times$ smaller coefficient against a fixed on-chip budget. Second, how many survivors are worth refining? We select that budget with a label-free rule that compares the returned top- k sets at a candidate budget against a generous reference on sampled workload queries. On six datasets from 768 to 3,072 dimensions and up to 17.8M vectors, six controls contradict the naive byte-count prediction: a table-free form holds least state and loses 30–52%; four- and eight-way splits hold fewer entries than the two-way split and lose up to 38%; packed loading moves identical bytes and gains 30–48%. Disassembling the emitted kernel turns that from an inference into a decomposition: with the table resident, query time is close to linear in issued instructions per candidate-subspace, and the granularity whose table does not fit sits 4–120% above that line although it issues the fewest instructions of all. The budget rule is worth $1.044\text{--}2.653\times$ steady-state for at most 0.0048 recall, repaid in a median of 1.75 batches, but its sampling risk is workload dependent. Against quantised IVF baselines JHQ owns the high-recall end at high dimension and indexes two datasets where the tested IVF-RaBitQ build path runs out of memory; graph-based CAGRA-int8 is $1.6\text{--}2.9\times$ faster below Recall@10=0.95, at $5\text{--}8\times$ the build time. The result is a bounded operating region, not a universal-fastest claim.

Keywords: Approximate nearest neighbour search · GPU · Quantisation · High-dimensional data · Adaptive query processing

1 Introduction

High-dimensional approximate nearest-neighbour (ANN) search is a core primitive in retrieval, recommendation, and vector databases. On a GPU, compressed

search is not governed by memory footprint alone: a representation can reduce bytes yet raise lookup pressure or instruction issue, and an implementation can move the same logical bytes and still accelerate. The systems problem is to map the algorithm’s structure to the hardware bottleneck rather than to assume that smaller is faster.

JHQ [3] is a useful case study because its hierarchy exposes two costs of different shape. After IVF routing, a compact primary code scores every routed candidate at $O(n_c M)$; a per-dimension residual code is then read only for the c_k survivors of primary filtering, at $O(c_k d)$ with $c_k \ll n_c$. JHQ also leaves the refinement factor α in $c_k = \alpha k$ externally specified. The GPU port therefore raises two coupled questions with different causes: *how should the primary representation execute*, and *how much of the expensive hierarchy should execute*?

Our answer to the first is representation-specific. A conventional 8-bit ADC table materialises 256 distances per subspace. JHQ builds each codeword as a Cartesian product of one-dimensional levels, and squared Euclidean distance is additive across them, so the same represented distance follows exactly from two 16-entry tables. This is not “smaller is better”. Full-table state grows as $256M$ entries while the two-way split grows as $32M$, an $8\times$ smaller coefficient against a fixed on-chip budget, but each extra split costs another lookup per candidate–subspace. Our answer to the second is a label-free workload policy: sample queries from the running workload, compare the top- k sets they return at a candidate budget against a generous reference, and take the smallest budget that leaves them unchanged. It needs neither ground-truth neighbours nor a learned difficulty model, at the cost of assuming a reasonably stable query distribution.

We evaluate on six datasets from 768 to 3,072 dimensions, up to 17.8M vectors, on one RTX 5090, and we test the alternatives we rejected rather than only the implementation we kept. Six controls contradict the byte-count prediction, most sharply between the four- and eight-way splits, which hold the same number of entries and differ only in how many lookups reach them. Static disassembly then measures the implicated variable directly, without a profiler: issued instructions per candidate–subspace are 4.4, 13.0, 25.0 and 50.4 at $G=1, 2, 4, 8$, query time is close to linear in them wherever the table is resident, and the one granularity whose table does not fit sits above that line by 4–120% while issuing the fewest instructions of any. The budget rule is worth $1.044\text{--}2.653\times$ in steady state for at most 0.0048 recall and repays its calibration in a median of 1.75 batches, while its sampling risk varies enough by workload that no single sample size is safe.

The competitive picture is bounded and we state it as such. Among quantised IVF methods JHQ leads through Recall@10=0.95 at large batch, owns the high-recall end at high dimension, and indexes two datasets where the tested cuVS IVF-RaBitQ build path runs out of memory. It is not the fastest system measured: graph-based CAGRA-int8 is $1.6\text{--}2.9\times$ faster below Recall@10=0.95 on every dataset where both run, at $5\text{--}8\times$ the build time and a comparable code budget.

Contributions. (i) A decomposition of the primary scan’s cost into an issue term and a residency term, each measured: six controlled comparisons in which fewer bytes or less state fails to predict throughput, and a static instruction count from the disassembled kernel against which the residency penalty appears as the residual. (ii) An exact Cartesian factorisation of JHQ’s primary distance table, derived from separability the codebook already has rather than from a different quantiser, together with the regime in which it pays. (iii) A label-free, predictor-free policy for JHQ’s residual budget, evaluated on gain, quality cost, amortisation, and sampling risk. (iv) An evaluation that reports where the compared systems are faster than ours and where our evidence does not generalise.

2 Preliminaries

JHQ represents each vector with a square orthogonal transform, a Cartesian primary codebook, and a per-dimension residual code [3]. We introduce only what later sections need.

For a database vector x and query q in $\mathbb{R}^d$, JHQ applies an orthogonal $Q \in \mathbb{R}^{d \times d}$: $y = Qx$, $q' = Qq$ (row-major batches use $Y = XQ^\top$). Q is square, so this is a change of coordinates, not dimensionality reduction, and it preserves Euclidean distance and hence the neighbour ordering.

The transformed vector is split into M subspaces of dimension D_s , with $d = MD_s$. Each primary subspace code has B bits and selects one of $K = 2^B$ codewords, and each codeword is a Cartesian product of D_s one-dimensional Lloyd–Max levels. With L levels per coordinate, $L = 2^{B/D_s}$, so the implementation admits only D_s dividing B : at $B=8$, $D_s \in \{1, 2, 4, 8\}$ gives $\{8, 4, 2, 1\}$ primary bits per coordinate. We use $D_s=8$, so each coordinate carries one primary bit and each subspace is one byte. This Cartesian construction is what Section 3.2 exploits.

The residual is $r = y - \hat{y}_{\text{primary}}$, the displacement from the primary reconstruction — *not* from an IVF centroid. IVF is an external routing layer that restricts the candidate set; it does not redefine what the residual quantiser represents. The residual code is per dimension, so at B_r bits it costs about dB_r bits per vector, and at $B_r=8$, $D_s=8$ the logical representation is about 1 primary plus 8 residual bits per dimension.

Routing yields a candidate set $C(q)$. The primary score D_P is evaluated for every routed candidate; JHQ keeps the c_k with the smallest primary scores and evaluates the hierarchical score D_H only for those, returning the top- k under D_H among them. We write $c_k = \alpha k$ for exposition; the implementation uses $c_k = \max(k, \lceil \alpha k \rceil)$, and since every evaluated $\alpha \geq 1$ the max only enforces the k -survivor floor. So α controls an irreversible filtering point: a neighbour dropped by primary ranking cannot be recovered by refinement, while routing can drop true neighbours before the primary stage at all. The two loss sources stay separate throughout.

3 GPU-Native JHQ

3.1 Execution and representation

We begin from the work JHQ requires after routing rather than from CUDA primitives. The primary stage is $O(n_c M)$ over compact codes and the residual stage $O(c_k d)$ over per-dimension codes, so three requirements follow: make primary access match candidate-parallel warp execution, keep the primary distance representation resident without replacing JHQ’s quantiser, and preserve selective residual access instead of fusing accurate scoring into the broad scan. The orthogonal transform, primary codebook, residual definition and hierarchical ordering remain JHQ’s; physical representation, execution and the external refinement policy change, and the scan uses a bounded per-thread top- αk selector whose deviation from exact selection Section 4.2 measures.

During the broad scan, warp lanes score different candidates while advancing through the same subspace m , so candidate-major $[N, M]$ storage has the warp read one byte per lane with stride M . We store the scan copy subspace-major, $[M, N]$, which makes those 32 bytes adjacent: the transpose aligns the contiguous memory dimension with the dimension the warp shares. It is standard practice, not a claim. Separately, $D_s=B=8$ makes four subspace codes a native 32-bit word that a packed scan loads once and unpacks in registers. After the layout fix the fetched sectors are already right, so packing reduces load instructions, addresses and loop iterations rather than logical bytes. We do not claim wider words are universally better; they add unpacking, dependencies and register pressure.

3.2 Cartesian factorisation of the primary table

The question is not how small the table can be but how much table state to materialise before saved residency is outweighed by added issued work. For subspace m and code c , let $T_m[c] = \|q'_m - C_m[c]\|_2^2$. Because $C_m[c]$ is a Cartesian product of one-dimensional levels and squared Euclidean distance is additive, any partition of the eight code decisions induces an exact sum of smaller tables. For the two-way partition, with $c_{hi} = c \gg 4$ and $c_{lo} = c \bmod 16$,

$$T_m[c] = T_m^{hi}[c_{hi}] + T_m^{lo}[c_{lo}]. \quad (1)$$

Each component holds 16 entries, so $G=2$ stores 32 per subspace and performs two lookups. Table construction falls by $16\times$, not the $8\times$ of resident entries: the full table evaluates $256D_s$ coordinate contributions per subspace and the split form $32(D_s/2) = 16D_s$. An equal G -way partition stores $G \cdot 2^{8/G}$ entries and needs G lookups per candidate-subspace, which states the trade-off analytically — state falls only until the table fits, while instruction demand keeps rising.

The residency target is narrower than the device’s shared-memory limit, because the table is not the only claimant. The scan kernel’s admission test is

$$\text{scan_base} + M \cdot G \cdot 2^{8/G} \cdot 4 \leq \text{smem_optin}, \quad \text{scan_base} = 40 \cdot \text{BLOCK bytes}, \quad (2)$$

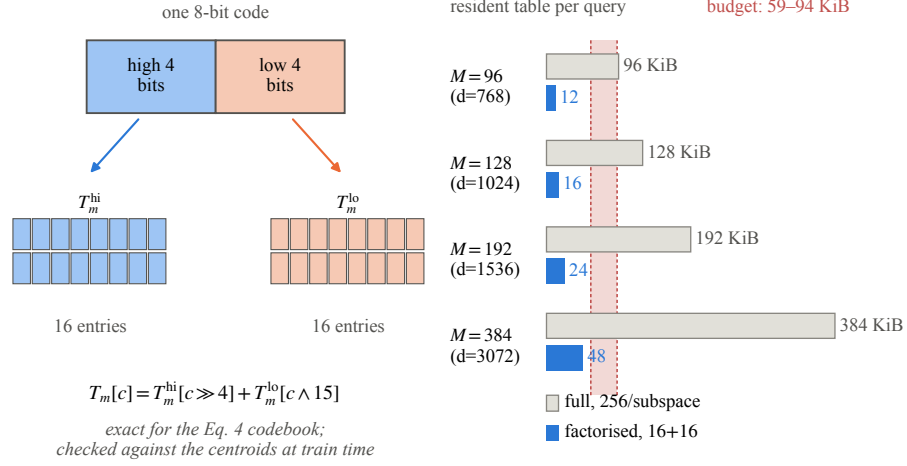


Fig. 1: Exact Cartesian factorisation of an 8-bit JHQ primary lookup table: 256 resident entries become two 16-entry tables. The band on the right is what Equation 2 leaves for the table across the admissible block sizes; every full table falls outside it and every factorised table inside.

with `scan_base` the per-thread candidate scratch claimed first. At the tested `smem_optin` = 101,376 B this leaves the table 94.0 KiB at `BLOCK`=128, falling to 59.0 KiB at `BLOCK`=1024. The full table is $M \cdot 256 \cdot 4$ bytes — 96 KiB at $M=96$ and 384 KiB at $M=384$ — so *neither* clears the budget at any admissible block size, $M=96$ missing the most generous by 2 KiB, while both factorised tables clear the least generous at 12 and 48 KiB (Figure 1). The full table is thus off chip at both M , and what grows with M is the cost of the lookups that miss, since the scan pays one per candidate per subspace. The prediction we test in Section 6.3 is therefore directional: reducing table state should help until the extra lookup work outweighs the residency it buys, and the crossing should move with M and with probe depth together.

This is not generic small-table ADC. The distinction from Quicker ADC [2] is provenance, not loss: it obtains small tables by changing the quantiser to narrower irregular sub-quantisers, whereas Equation 1 holds for JHQ’s existing 8-bit codebook without touching a codeword. Exactness is in real arithmetic; it promises no bitwise-identical floating-point association or tie order.

3.3 Selective residual refinement, and what it leaves open

Reading the per-dimension residual for every routed candidate would turn an $O(c_k d)$ stage into work closer to $O(n_c d)$, so only the c_k primary survivors trigger residual access. Fusing the two phases is a plausible alternative — it removes a round trip and a kernel boundary — and Section 6.4 reports why we reject it.

The representation says how to execute each unit of JHQ work cheaply, not how many expensive units are useful. A global α large enough for a hard workload

wastes the residual stage on an easy one; a small one filters irreversibly. Section 4 therefore treats α as a workload control problem, not another kernel constant.

4 Adaptive Hierarchical Refinement

4.1 Criterion

Original JHQ leaves α externally specified, and the useful budget is workload dependent: at $nprobe=128$ openai3-3072 is flat from $\alpha=4$ over the measured grid while arxiv-768 is still improving at $\alpha=200$, a censored upper endpoint rather than a plateau. A single global constant therefore either wastes $O(\alpha kd)$ residual work on easy workloads or filters candidates before accurate refinement on hard ones. We calibrate a workload-level operating point rather than predict α per query, which keeps the control path label-free and amortisable.

Let Q_s be S sampled queries from the current workload and $R_{\max}(q)$ the top- k ID set returned under a generous $\alpha_{\max}$. For a tested α with returned set $R_\alpha(q)$, define the total slot disagreement

$$D(\alpha) = \sum_{q \in Q_s} [k - |R_\alpha(q) \cap R_{\max}(q)|], \quad (3)$$

and select the smallest tested-grid α with $D(\alpha) \leq \epsilon$. The tolerance ϵ is a non-negative integer over the whole sample, not a per-query fraction. Set membership asks whether more residual work changes *which* neighbours are returned while ignoring harmless permutations of the same IDs. $S=32$ and $\epsilon=1$ are empirical defaults, not constants. Ground-truth recall is what we care about but is unavailable online, and a learned predictor costs labels, maintenance and drift behaviour outside this scope. Output stability is observable from the running index, but cannot recover routing losses, certify recall, or see past $\alpha_{\max}$; Section 6.5 validates it against ground truth offline rather than treating self-agreement as a guarantee.

4.2 Monotonicity and bisection

We evaluate $\alpha_{\max}$ once to build the reference, then bisect the grid. Bisection needs a monotone acceptance predicate, so we state the condition under which one exists.

Proposition 1 (monotonicity of disagreement). *Let $\alpha_1 \leq \alpha_2 \leq \alpha_{\max}$. Under exact top- c_k primary selection with c_k non-decreasing in α , a fixed refined distance, and a deterministic total tie rule, $D(\alpha_1) \geq D(\alpha_2) \geq D(\alpha_{\max}) = 0$.*

Proof. Exact top- c_k selection nests the survivor sets, $S_{\alpha_1} \subseteq S_{\alpha_2} \subseteq S_{\alpha_{\max}}$. Order $S_{\alpha_{\max}}$ by the refined distance with the deterministic tie key and let $R_{\max}$ be its first k elements. Any element of $R_{\max}$ present in S_α also appears in top- $k(S_\alpha)$, since fewer than k elements of $S_{\alpha_{\max}}$ precede it and hence fewer than k of the subset S_α do. So $R_\alpha \cap R_{\max} = S_\alpha \cap R_{\max}$, which can only grow with α . $\square$

Implementation caveat and measured deviation. The production scan uses a bounded per-thread selector with $K_LOCAL=4$, so a thread can discard a globally top- c_k item when more than K_LOCAL of them fall in its stride class, and theorem-level nestedness is not asserted for the deployed kernel. We measure the gap rather than hide it behind aggregate recall: with the index pinned, vogue-768 at $nprobe=128$ and $BLOCK=512$, raising K_LOCAL from 4 to 8 changes 11 of 10,000 returned ID positions, touches 4 of 1,000 queries, and gives 1 query a different result set. $BLOCK=512$ is deliberately the harder collision regime, and $K_LOCAL=8$ is register-infeasible at the production $BLOCK=1024$.

The consequence for bisection should be stated precisely. Across all 15 calibration traces that probe two or more grid points, $D(\alpha)$ is non-increasing at the points the search visited. That is evidence, not a proof: points the search never visited are untested, so we do not claim bisection returns the global minimum acceptable α on the grid, only that its stopping behaviour is conservative with respect to the sampled criterion. Floating-point ties are a separate matter — the proposition assumes a deterministic total order, not bitwise-identical association.

Sampled parameter tuning is not itself new: FAISS [5] exposes parameter-space exploration and Li et al. [6] learn adaptive termination. Our contribution is narrower — a predictor-free top- k stability signal applied to JHQ’s external residual budget, with its proxy error and economics quantified.

4.3 Calibration economics

With T_{cal} the calibration time and T_{fixed} , T_{rule} the per-batch times before and after selection, the break-even batch count is $B^* = T_{cal}/(T_{fixed} - T_{rule})$ whenever $T_{rule} < T_{fixed}$. A more expensive calibration is rational only when the budget saves enough steady-state work and the operating point persists; Section 6.5 reports both terms. Reuse assumes the query distribution, routing configuration, k and index stay stable — the price of workload-level rather than per-query adaptation. Drift is unmeasured: recalibrating every H batches adds T_{cal}/H per batch, but choosing H is a drift-detection problem outside this evidence. The timer starts after sample copying, so the cost is algorithm-level, not a full control-plane cost.

5 Related Work

We organise related work around the novelty threats that matter for this paper rather than as a catalogue.

Quantisation and JHQ. Product Quantization (PQ) [4] established Cartesian decomposition across subspaces and asymmetric distance computation. JHQ [3] targets high-dimensional ANN with an orthogonal transform, a Cartesian primary quantiser, and hierarchical residual refinement. We do not claim JHQ’s hierarchy or orthogonal transform as contributions; our work begins from that representation and asks how its specific structure should execute on a GPU.

Small-table distance evaluation. PQ Fast Scan [1] showed that cache residency alone is insufficient for fast ADC and reorganised PQ scanning around small tables and SIMD-register lookups. Quicker ADC [2] generalised this direction with irregular product quantisers and split tables for wider code widths. These works make it essential not to claim novelty merely from “smaller LUTs.” The distinction is in provenance, not in loss: Quicker ADC obtains small tables by changing the quantiser, replacing 8-bit sub-quantisers with narrower irregular ones whose tables are small by construction. Our table is small because JHQ’s existing 8-bit Cartesian subspace code already separates exactly (Equation 1), so we preserve the represented codebook and derive an identity for its distance table rather than choosing a different one. We also show empirically that a smaller table can be slower once extra lookup instructions dominate, most sharply between $G=4$ and $G=8$, which occupy the same nominal footprint. Our claim is about exploiting a particular representation, not about inventing small-table ADC in general.

Adaptive ANN control. ANN libraries such as FAISS [5] commonly expose parameter sweeps and tuning utilities, while learned approaches can adapt search termination to the query [6]. We therefore do not claim sampling or parameter selection as new. Our output-stability rule differs in its signal and deployment assumptions: it needs no labelled nearest neighbours and no learned predictor, compares the system’s own returned top- k sets to a generous-budget reference, and chooses the externally specified JHQ refinement budget. The evaluation separately quantifies selection error and amortisation, which are necessary because a label-free proxy is not a recall guarantee.

GPU ANN systems. CAGRA [7] represents the graph-based GPU frontier, and cuVS provides production implementations of CAGRA, IVF-PQ, and IVF-RaBitQ. GPU IVF-RaBitQ [8] combines inverted-file routing with low-bit RaBitQ codes and reports a strong recall-throughput/build/storage trade-off. These systems are not uniformly weaker than JHQ: our matched experiments show regime crossings in the high-recall tail and with batch size. This is why we report nondominated frontiers and configuration-specific build failures rather than a single aggregate speedup.

6 Evaluation

6.1 Setup and protocol

All GPU systems run on one NVIDIA RTX 5090: 32,607 MiB device memory, 170 SMs, a 101,376-byte opt-in shared-memory limit per block; the host has 208 logical cores and 754 GB RAM. Datasets are vogue-768 ($1.0\text{M} \times 768$), arxiv-768 ($2.25\text{M} \times 768$), bge-m3 ($10.1\text{M} \times 1024$), stella-trec24 ($17.8\text{M} \times 1024$), openai3-1536 ($1.0\text{M} \times 1536$) and openai3-3072 ($1.0\text{M} \times 3072$).

We report Recall@10 against true top-10 ground truth at $k=10$; because c_k scales with αk , other k are not established. Timing runs host query input to host result output, and frontiers use steady-state throughput at batch 1,024 —

Table 1: Fastest system at each matched recall, batch 1,024, with its margin over JHQ. `int8` and `fp32` are cuVS CAGRA, RaBitQ is cuVS IVF-RaBitQ; “–” is a recall no two systems both reach. JHQ is fastest in 4 of 26 cells, all at $\text{Recall@10} \geq 0.97$.

dataset	fastest system at Recall@10					
	0.90	0.93	0.95	0.97	0.98	0.99
vogue-768	int8 2.7×	int8 2.0×	fp32 1.1×	RaBitQ 1.0×	RaBitQ 1.1×	JHQ
arxiv-768	int8 2.9×	int8 2.9×	int8 2.5×	fp32 1.4×	fp32 1.4×	RaBitQ
bge-m3	int8 2.4×	int8 1.7×	–	–	–	–
stella-trec24	–	–	–	int8 2.6×	–	–
openai3-1536	int8 1.9×	int8 1.6×	int8 1.3×	JHQ	JHQ	–
openai3-3072	int8 2.4×	int8 2.1×	int8 1.6×	JHQ	RaBitQ 1.1×	RaBitQ 1.4×

the whole query set in one call — which Section 6.6 shows is a consequential choice. Occupancy statements are analytical resource-budget calculations: Nsight Compute counters were unavailable in the container (`ERR_NVGPUCTRPERM`), so we report none measured. Two recall noise floors are kept distinct: about 10^{-3} across builds in the documented cold-start experiment, and $\approx 10^{-4}$ for paired comparisons on one index from top- c_k tie breaking. Fronts bucket recall at 10^{-3} so a tie-break-sized difference cannot buy a point a place, and every quoted ratio is the median of its one to three repeats. We do not enlarge batches by duplicating the roughly 1k available queries: duplication alone raises throughput 4–26%, so such a batch would measure an artefact.

Baselines are the cuVS implementations of IVF-RaBitQ [8], IVF-PQ, and CAGRA [7] in `fp32` and `int8`. Our primary comparison line is the quantised IVF family, which shares JHQ’s routing structure and interface; CAGRA is a graph index with a different build-cost profile and we report it as context, in both variants, without excluding the one that wins. JHQ runs use dense coarse training, $\text{JHQ_N_TRAIN} = 39 \cdot n_{\text{list}}$; we mix in no under-trained points and union no fronts across binaries. The CPU JHQ baseline runs the same index parameters on the same host with threads pinned (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and its own $(\alpha, n_{\text{probe}})$ sweep, sharing the training procedure but not a code-book, with one measurement per cell.

6.2 RQ1: where does JHQ actually lead?

Figure 2 gives the measured frontiers and Table 1 reduces them to a single question: at each matched recall, which system is fastest? JHQ owns 4 of the 26 comparable cells, and all four are at $\text{Recall@10} \geq 0.97$. Below 0.95 CAGRA-int8 leads on every dataset where both run, by 1.6–2.9×. We state this rather than scope it away, because it bounds the claim precisely: JHQ’s region is the high-recall end, most clearly at high dimension, where openai3-1536 leads at 0.97 and 0.98 and openai3-3072 at 0.97.

Two things beside the table complete the picture. On bge-m3 and stella-trec24 the tested cuVS IVF-RaBitQ host-input build path fails with an out-of-memory

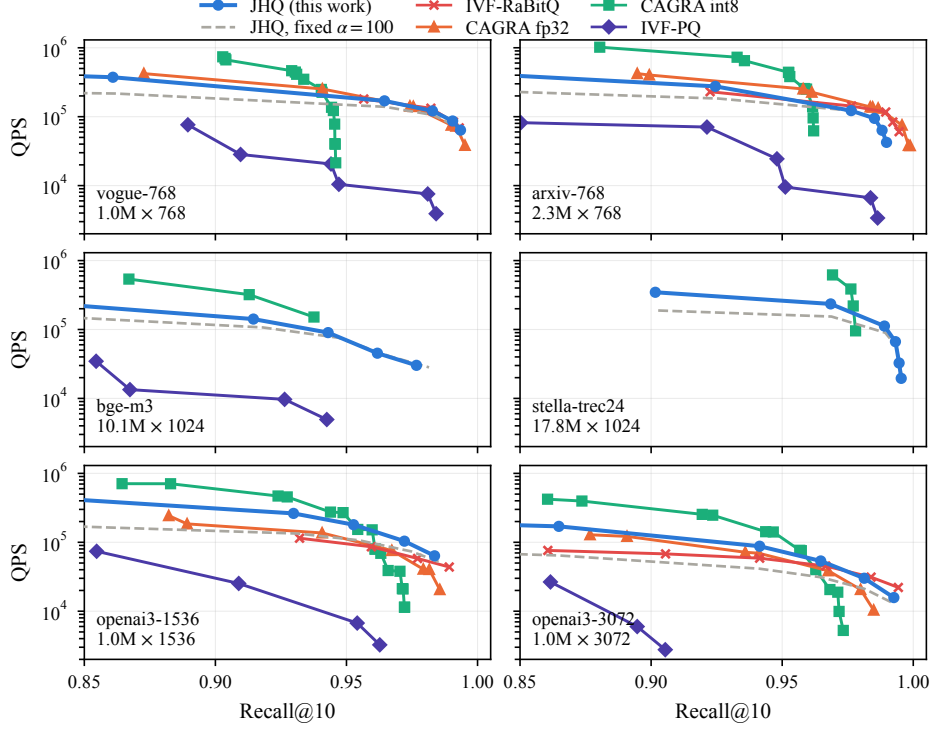


Fig. 2: Measured Recall@10-QPS frontiers, batch 1,024. Four datasets carry complete JHQ/IVF-RaBitQ curves; the two largest show the baselines that could be built and searched on the tested card. Sweep endpoints are measured endpoints, not architectural recall ceilings.

allocation after entering streaming construction and CAGRA fp32 does not fit the card, while JHQ indexes both — a statement about the measured library and configuration paths on this device, not about those algorithms in general. And the throughput ordering has a build-time counterpart: JHQ builds in 1.1–14.3s against 9–75s for CAGRA int8. JHQ therefore trades search throughput below 0.95 for build time and for indexability at a code budget — about 9 bits per dimension — that is not smaller than int8’s.

6.3 RQ2: does the design prediction hold?

Section 3.2 predicted that reducing table state helps only until extra lookups outweigh the residency saved. Figure 3 varies G with the represented distance fixed. If state alone governed throughput, $G=4$ and $G=8$ should win, since they store fewer entries than $G=2$. They do not: $G=2$ is never beaten across the eight configurations, with one exact tie against $G=1$, and throughput falls as lookups per candidate rise. The sharpest form is internal to the losers. $G \cdot 2^{8/G}$ is 16 for both $G=4$ and $G=8$, so they occupy the same nominal table footprint and differ in the number of lookups and the address arithmetic those lookups need; the

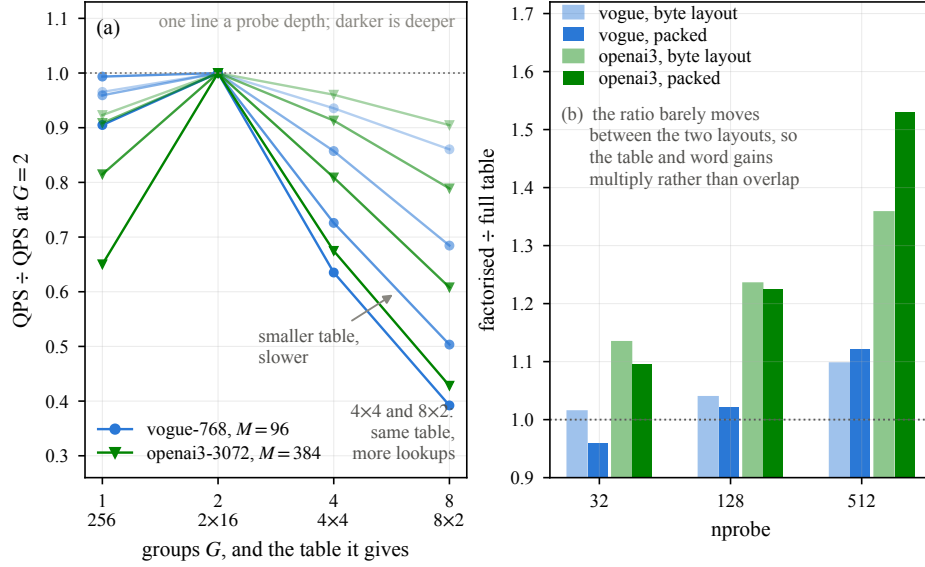


Fig. 3: (a) $G=2$ is never beaten although $G=4$ and $G=8$ store fewer entries and occupy the same footprint as each other. (b) Factorisation matters far more at $M=384$ than $M=96$ and stays complementary to packed loading; panel (b) is the two-by-two layout phase at $nprobe \in \{32, 128, 512\}$, while the $nprobe=8$ point quoted above is panel (a)’s lightest probe-depth line.

eight-way form is 5.8% slower at $nprobe=8$ and 38.3% slower at 512. Residency is held fixed and issued work rises — which is why this pair, rather than the table-free control, separates the two effects most cleanly.

The regime change with M appears as predicted. Against the full table, factorisation is worth 0–11% at $M=96$ and 8–53% at $M=384$, and the gain rises monotonically with probe depth in both: on openai3-3072 +8.3%, +9.8%, +22.6% and +53.4% at $nprobe = 8, 32, 128, 512$. We report the trend rather than a flat interval because the trend is the mechanism — a table that misses is paid once per candidate per subspace, so its cost grows with M and with the candidates scanned together. The failure edge is kept too: at $M=96$, $nprobe=32$ on the packed layout, factorisation loses 4.2% against the full table in the slowest repeat.

6.4 RQ3: can the byte-count explanation be rejected?

Table 2 collects the controls that vary bytes and issued work independently. Removing almost all table state loses 30–52% even though it raises the resident-block count the shared-memory budget implies — so neither table residency nor attainable occupancy alone is the bottleneck. Moving identical bytes in 32-bit words gains 30–48%. Giving each thread a private probe cursor removes redundant inner-loop work for up to +148%; it is an implementation correction, not a contribution, and its value here is diagnostic. Fusing residual refinement

Table 2: Controls that separate bytes moved from work issued. Every row holds the represented distance fixed.

change	effect on throughput	what it rules out
table-free exact distance	−30 to −52%	least state is fastest
$G=8$ against $G=4$	−5.8 to −38.3%	same footprint, same speed
packed 32-bit loads	+30 to +48%	bytes moved predict speed
private probe cursor	+2.5 to +148%	loop work is negligible
fused residual refinement	−14.2%, −14.8%	fewer kernels is better
query duplication $D=8$	+4 to +26%	reuse alone drives the scan

removes an intermediate round trip and still loses, because the fused kernel carries both phases’ shared-memory and register footprints for the whole scan. LUT construction is 0.2–2.3% of a batch, which rules out faster table building as the explanation for factorisation.

These controls rule out competing explanations but do not measure the variable they implicate. Static instruction counts do not need a profiler, so we disassemble the emitted kernel. `cuobjdump -sass` on `scan_ivf_coalesced_kernel` gives 42, 84 and 168 generic loads at $G=1, 2$ and 4 — one, two and four lookups across 42 statically unrolled subspace iterations — and 144 at $G=8$, which is eight lookups over 18 iterations, the compiler having unrolled less because the body grew. Floating-point adds per iteration are $G+1$ at every G , an independent check that the normalisation is right. Counting the lookup and the work that feeds it, issued instructions per candidate–subspace are 4.4, 13.0, 25.0 and 50.4.

That is an axis the controls did not have, and it separates the two costs Section 3.2 trades between (Figure 4). Where the table is resident — $G=2, 4, 8$ — query time is close to linear in issued work, and the slope is the issue cost. $G=1$ issues the least work of all four and is nonetheless slower than the line predicts, by 8, 4, 33 and 75% on vogue-768 and 11, 16, 44 and 120% on openai3-3072 at $nprobe = 8, 32, 128$ and 512. That gap is the residency cost, it is the one thing the instruction line cannot explain, and it grows with probe depth and with M exactly as “a table that misses is paid once per candidate per subspace” predicts. $G=2$ wins because it is the only granularity that pays little of either cost. We still measure one architecture, so cross-GPU validation stays open, but the mechanism itself is no longer an inference from a family of controls.

6.5 RQ4: what does the budget rule cost and return?

Three things must hold together for the policy to be usable. *Gain*: across 32 configurations where the rule changes the budget, steady-state throughput improves $1.044\text{--}2.653\times$. *Quality cost*: the rule gives up at most 0.0048 recall over 36 configurations, so the gain is not free. *Amortisation*: payback ranges 0.5–21.0 batches with median 1.75; large payback near unit gain is partly definitional through the denominator, so we do not present that correlation as a mechanism.

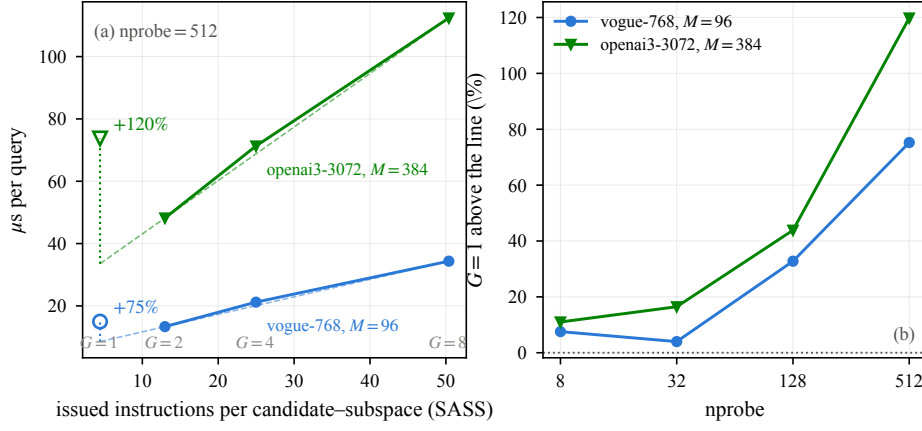


Fig. 4: The two costs, measured separately. (a) With the table resident ($G=2, 4, 8$) query time is close to linear in issued instructions per candidate-subspace, counted from the disassembled kernel; $G=1$ issues the fewest and still sits above that line, and the gap is the residency cost. (b) That gap grows with probe depth and is larger at $M=384$ than $M=96$.

The fourth thing is sampling risk, and it is where the policy is weakest. In 2,000 held-out resampled draws, $S=32$ keeps held-out loss within 10^{-3} in 76% of draws for *openai3-3072* but only 27% for *vogue-768*, whose mean loss at $S=32$ is 0.0033 with a 95th percentile of 0.0110. *Openai3-3072* reaches 98% at $S=64$; *vogue* still has 11% of draws outside the threshold at $S=128$. $S=32$ is therefore not a universal knee, and a deployment must pick S against a stated risk target. Tolerance behaves similarly — zero disagreeing slots leaves about 26% of attainable throughput unclaimed, two costs about 0.0035 recall — and at $nprobe=128$ the rule recovers the swept α in three of four configurations, the fourth censored by $\alpha_{\max}$ rather than solved. The $nprobe=512$ full sweep is unavailable and is not inferred.

6.6 RQ5: batch size and the CPU reference

Table 3 gives matched-recall ratios against IVF-RaBitQ at four batch sizes. Two readings matter: on *openai3-3072* JHQ leads at every measured point, and on *vogue-768* it crosses parity between batch 128 and 512. Batch size can therefore change the ordering, and the frontier ratios of Section 6.2 should be read as the batch-1,024 column of this sweep rather than as batch-independent summaries. The *openai3-3072* batch-1,024, $R=0.97$ cell is excluded: its RaBitQ anchor is the one matched point that failed the project’s monotonicity check.

The CPU reference answers a different question: what the port buys over CPU JHQ at the same index parameters. Sweeping α and $nprobe$ on both sides, the GPU reaches 52–54 $\times$ the CPU envelope on *vogue-768* across Recall@10 of 0.9645 to 0.9905, and 75–77 $\times$ on *openai3-3072* from 0.9414 to 0.9813; pinning $\alpha=100$ on both sides gives 43–48 $\times$ and 35–54 $\times$. Above Recall@10 of 0.9911 and

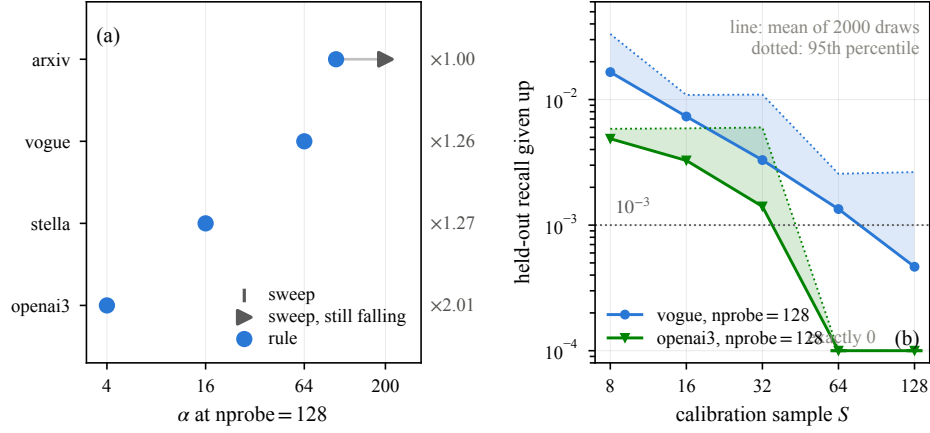


Fig. 5: Calibration validation. (a) Selected α against the exhaustive sweep where a same-operating-point sweep exists, annotated with the throughput the selection gains; (b) held-out resampling, showing that sample-size risk is workload dependent; (c) tolerance changes both the selected budget and the quality it costs.

Table 3: Matched-recall JHQ/IVF-RaBitQ throughput, by batch size, at Recall@10 of 0.90 and 0.95. Each system’s curve is interpolated at the target recall.

	openai3-3072		vogue-768	
batch	0.90	0.95	0.90	0.95
32	2.23	1.34	0.68	0.55
128	3.19	2.14	0.99	0.92
512	2.30	1.51	1.06	1.01
1024	1.89	1.41	1.09	1.33

0.9900 respectively the CPU sweep has no points, and those GPU operating points are reported as outside its range rather than divided by the CPU’s last measurement. Three limits apply. The GPU side of the vogue comparison is the rule’s chosen α at each $nprobe$, not an exhaustive α sweep. CPU rows below $nprobe=128$ are dropped, because 1,000 queries complete fast enough there that thread start-up dominates the timer and measured throughput rises with $nprobe$; the recall column is unaffected. And each cell is measured once, so the ratios are order-of-magnitude figures — the envelope keeps the fastest point per recall bucket, which biases the CPU side upward and so does not inflate the ratio, but with one measurement per cell we do not claim a statistical bound.

6.7 Construction and memory

Build times are 1.1–14.3s for JHQ, 9–75s for CAGRA int8, 6.4–43s for IVF-PQ and 4.5–14.0s for IVF-RaBitQ where that path builds — tested configurations,

not intrinsic bounds. Accounting must respect caching: training can be cached while add is not, so taking maxima independently and summing them fabricates a time that never occurred. On memory we claim no advantage: in the four datasets with measured VRAM IVF-RaBitQ uses 22–39% less than JHQ, and JHQ’s budget is not smaller than int8’s. JHQ occupies a different memory–accuracy–throughput point and still indexes the two largest datasets where the compared paths fail. The accounting carries an unattributed component that should not be relabelled as allocator overhead without direct measurement.

7 Conclusion

We asked how JHQ’s primary representation should execute on a GPU and how much of its hierarchy is worth executing. For the first, the Cartesian codebook admits an exact two-table factorisation whose state grows as $32M$ against $256M$, and the measurements show the payoff rising with both M and probe depth while still-smaller splits lose to their extra lookups. Disassembling the kernel separates the two terms: query time is close to linear in issued instructions per candidate–subspace wherever the table is resident, and the granularity whose table does not fit sits 4–120% above that line while issuing the fewest instructions — so the target is a residency budget, not minimum bytes. For the second, a label-free stability rule buys $1.044\text{--}2.653\times$ for at most 0.0048 recall and repays itself in a median of 1.75 batches, with sampling risk that varies enough by workload that the sample size must be chosen against a stated target. The region this defines is bounded: among quantised IVF methods JHQ leads to Recall@10=0.95 at large batch and owns the high-recall end at high dimension, while CAGRA-int8 is faster below that at several times the build cost. Remaining gaps are k beyond 10, workload drift, and cross-GPU validation.

References

1. André, F., Kermarrec, A.M., Scouarnec, N.L.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.* **9**(4), 288–299 (2015). <https://doi.org/10.14778/2856318.2856324>
2. André, F., Kermarrec, A.M., Scouarnec, N.L.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.* **43**(5), 1666–1677 (2021). <https://doi.org/10.1109/TPAMI.2019.2952606>
3. Han, J., Zhang, M., Trajcevski, G.: JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.* **19**(7), 1530–1543 (2026). <https://doi.org/10.14778/3801059.3801067>
4. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.* **33**(1), 117–128 (2011). <https://doi.org/10.1109/TPAMI.2010.57>
5. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *arXiv:1702.08734* (2017)

6. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: SIGMOD. pp. 2539–2554 (2020). <https://doi.org/10.1145/3318464.3380600>
7. Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., Wang, Y.: CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In: IEEE ICDE (2024). <https://doi.org/10.1109/ICDE60146.2024.00323>
8. Shi, J., Gao, J., Xia, J., Feher, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. Proc. VLDB Endow. **19**(11), 3454–3467 (2026). <https://doi.org/10.14778/3836663.3836701>